LAPORAN PRATIKUM

ALGORITMA DAN STRUKTUR DATA

Function* dan *Array

OLEH:

NAMA : Adinda Riesya Putri

NIM : 2025573010039

KELAS : TI 1D

DOSEN : M.Reza Zulman S.S.T., M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026

LEMBARAN PENGESAHAN

No Praktikum	:	03/TIK/TI 1D/STRUKTUR DATA DAN ALGORITMA/2026
Nama Praktikum	:	Praktikum Struktur Data dan Algoritma
Pembuat Laporan	:	Adinda Riesya Putri
NIM	:	2025573010039
Kelas	:	TI-1D
Jurusan	:	Teknologi Informasi dan komputer
Prodi	:	Teknik Informatika
Tanggal Praktikum	:	9 April 2026
Tanggal Penyerahan Laporan	:	15 April 2026

Mengetahui

Dosen Pengajar,

M.Reza Zulman S.S.T., M.Sc

NIP: 199205012022031005

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bagian dari tugas dalam mata kuliah Praktikum Algoritma dan Struktur Data yang bertujuan untuk memberikan pemahaman dalam menyiapkan hal-hal yang dibutuhkan dalam praktikum.

Dalam laporan ini, membahas tentang *function*s dan *array* yang ada dalam bahasa javascript dan yang digunakan dalam struktur data serta rekursi. Laporan ini merupakan salah satu tugas praktikum dari mata kuliah Praktikum Algoritma dan Struktur Data.

Saya menyadari bahwa laporan ini tidak akan terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih yang sebesar-besarnya kepada dosen pembimbing, teman-teman, serta pihak-pihak yang telah memberikan bantuan baik secara langsung maupun tidak langsung dalam penyelesaian laporan ini.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi semua pihak yang membacanya. Kritik dan saran yang konstruktif sangat diharapkan untuk penyempurnaan laporan ini di masa mendatang.

Lhokseumawe 2 Maret 2026

Adinda Riesya Putri

Nim. 2025573010039

Daftar Isi

KATA PENGANTAR ..... **ii**

Daftar Isi ..... **iii**

BAB I PENDAHULUAN **1**

1.1 LATAR BELAKANG 1

1.2 TUJUAN PRAKTIKUM 1

1.3 RUMUSAN MASALAH 2

BAB II LANDASAN TEORI **3**

2.1 *FUNCTIONS* 3

2.2 ARRAY 3

2.3 REKURSI 4

2.5 TAIL RECURSION 6

BAB III LANGKAH LANGKAH PRAKTIKUM **7**

3.1 Alat dan Bahan 7

3.2 Pembuatan *File* dan *Folder* Menggunakan *Git* Bash 7

3.2 Percobaan dan Latihan 8

3.2.1 *Function* Dasar. 8

3.2.2 *Arrow Function* dan *Callback*. 10

3.2.3 *Array* Dasar. 11

3.2.4 *Array Method*. 12

3.2.5 Rekursi. 14

3.3 Tugas 15

3.3.1 Sistem Nilai Mahasiswa 15

3.4 Mengirim Ke *GitHub* 16

BAB IV PEMBAHASAN DAN ANALISA **18**

BAB V PENUTUP **38**

5.1 Kesimpulan 38

5.2 Saran 38

DAFTAR PUSTAKA **39**

BAB I
PENDAHULUAN

1.1 LATAR BELAKANG

Dalam dunia pemrograman, struktur data dan algoritma merupakan komponen yang penting dalam pengolahan data secara efisien. Struktur data digunakan untuk mengatur dan menyimpan data sehingga dapat digunakan secara optimal dalam suatu program. Hampir semua aplikasi komputer menggunakan struktur data untuk mengelola data dalam jumlah besar dan kompleks. Karena itulah pemahaman mengenai struktur data menjadi sangat penting bagi mahasiswa di bidang teknologi informasi.

Salah satu struktur data dasar yang sering digunakan adalah *array*. *Array* sendiri merupakan kumpulan elemen data dengan tipe yang sama yang disimpan dalam satu variabel dengan indeks tertentu. *Array* memudahkan programmer dalam menyimpan dan mengakses banyak data secara efisien. Operasi dasar *array* meliputi traversal, insertion, deletion, searching dan updating.

Selain *array*, konsep rekursi juga merupakan teknik penting dalam pemrograman. Rekursi adalah metode pemrograman dimana suatu fungsi memanggil diri nya sendiri untuk menyelesaikan masalah yang lebih kecil hingga mencapai kondisi dasar. Teknik ini digunakan dalam penyelesaian masalah seperti faktorial, deret fibonacci, dan algoritma divide and conquer.

Dalam implementasi rekursi, terdapat konsep call stack yang digunakan untuk menyimpan informasi fungsi yang sedang berjalan.call back berfungsi untuk mengatur pemanggilan fungsi yang secara berurutan hingga rekursi selesai dijalankan. Terdapat juga *tail recursion* yang merupakan bentuk rekursi dimana pemanggilan fungsi rekursi berada pada baris terakhir sehingga lebih efisien dan dapat dioptimalkan oleh compiler.

1.2 TUJUAN PRAKTIKUM

Ada pula tujuan praktikum kali ini sebagai berikut:

1. Memahami konsep *array* pada struktur data
2. Memahami rekursi dalam pemrograman.
3. Memahami *function* dalam *array*.

1.3 RUMUSAN MASALAH

Ada pula rumusan masalah dalam praktikum ini:

1. Apa yang dimaksud dengan *array*?
2. Apa yang dimaksud dengan rekursi dalam pemrograman?
3. Apa yang dimaksud dengan *function* dalam *array*?
4. Bagaimana implementasi *function*, *array* dan rekursi dalam pemrograman?

BAB II

LANDASAN TEORI

2.1 *FUNCTIONS*

*Function*s merupakan sebuah blok kode yang diberi nama dan dapat dipanggil/dieksekusi kapan saja dan berulang kali. Prinsip utama dalam pemrograman adalah Don't Repeat Yourself (DRY), yaitu jangan mengulang kodingan sebelum nya. Jadi daripada mengetik ulang kodingan tersebut lebih baik nya kita membuat kodingan itu di dalam sebuah *function*.

Di dalam *function* terdapat sebuah

1. Parameter

   Sebuah variabel input yang diterima *function* dari bagian luar. Didefinisikan di dalam kurung ()
2. *Return Value*

   Nilai yang dikembalikan *function* setelah selesai dieksekusi, menggunakan kata kunci *return*.
3. *Scope*

   Sebuah variabel yang dideklarasikan di dalam *function* hanya bisa diakses di dalam *function* itu saja (local). Kalau variabel di luar *function* disebut dengan *global function*.

2.2 ARRAY

Array adalah suatu struktur data yang digunakan untuk menyimpan sekumpulan data dengan tipe yang sama dalam satu variabel. *Array* memiliki jumlah elemen tetap dan setiap elemen dapat diakses menggunakan indeks tertentu. Struktur data *array* banyak digunakan dalam berbagai algoritma karena memudahkan [penyimpanan dan pengolahan data secara terstruktur.

Di dalam *array* terdapat beberapa istilah yang penting, seperti:

1. Element : data yang disimpan dalam *array*.
2. Index : posisi elemen dalam *array*.
3. Ukuran *array* : jumlah maksimal elemen yang dapat disimpan dalam *array*.

Array juga memiliki beberapa operasi dasar yang sering digunakan, yaitu:

1. Traversal

   Proses mengakses seluruh elemen *array* satu per satu untuk ditampilkan atau diproses.
2. Insertion

   Proses menambahkan elemen baru ke dalam *array* pada posisi tertentu.
3. Deletion

   Proses menghapus elemen tertentu dari *array*.
4. Searching

   Proses mencari elemen tertentu dalam *array* berdasarkan indeks atau nilai.
5. Update

   Proses mengubah nilai elemen pada indeks tertentu.

2.3 REKURSI

Rekursi adalah sebuah teknik di mana sebuah *function* memanggil dirinya sendiri untuk memecahkan masalah yang lebih kecil secara bertahap, hingga mencapai kondisi berhenti. Setiap *function* rekursif harus memiliki dua bagian, terdiri dari

1. Base Case/ Basis Kasus (kondisi berhenti)

   Kondisi di mana *function* tidak lagi memanggil diri nya sendiri. Tanpa adanya basis kasus fungsi rekursi akan terus berjalan tanpa berhenti dan dapat menyebabkan stack overflow, yaitu dimana kondisi memori komputer habis karena terlalu banyak pemanggilan fungsi.
2. Recursive Case

   Bagian di mana *function* memanggil dirinya sendiri dengan input yang lebih kecil/sederhana. Rekursi cocok digunakan pada masalah yang memiliki pola berulang seperti struktur data pohon, grafik, maupun metode divide dan conquer. Rekursi dapat diterapkan pada pemrograman seperti faktorial, deret fibonacci, pencarian biner, dan sebagainya.

Rekursi sangat berguna, tetapi tidaklah gratis. Setiap pemanggilan *function* menyimpan data di call stack. Rekursi yang terlalu dalam/ ribuan level bisa menyebabkan error Maximum call stack size exceeded. Untuk kasus seperti ini, solusinya adalah menggunakan iterasi for/while atau teknik *tail recursion*.

2.4 CALL STACK

Call Stack adalah mekanisme inti yang digunakan oleh JavaScript untuk mengatur dan melacak eksekusi fungsi. Call Stack menyimpan urutan pemanggilan fungsi dengan urutan eksekusinya. Mekanisme ini membantu menentukan fungsi mana yang sedang dijalankan saat ini dan fungsi apa yang akan dijalankan berikutnya.

JavaScript berjalan dalam lingkungan single-threaded, yang berarti hanya dapat menjalankan satu operasi pada satu waktu. Call stack merupakan bagian dari JavaScript yang digunakan untuk melacak proses eksekusi program seperti pemanggilan fungsi, eksekusi fungsi, pemanggilan bersarang, pengembalian fungsi, dan penyelesaian program.

1. Pemanggilan Fungsi

   Ketika sebuah fungsi dipanggil, fungsi tersebut akan dimasukkan ke dalam Call Stack dan tetap berada di sana hingga eksekusinya selesai.

2. Eksekusi Fungsi

   Mesin JavaScript menjalankan kode fungsi hingga selesai atau hingga fungsi tersebut memanggil fungsi lain.

3. Pemanggilan Bersarang

   Jika sebuah fungsi memanggil fungsi lain, maka fungsi itu akan dimasukkan ke dalam stack, sementara fungsi sebelumnya menunggu.

4. Pengembalian Fungsi

   Setelah eksekusi selesai, fungsi akan dikeluarkan dari stack dan kontrol kembali ke fungsi sebelumnya.

5. Penyelesaian Program

   Proses ini terus berlanjut hingga stack kosong, yang menandakan bahwa program telah selesai dijalankan.

2.5 TAIL RECURSION

Fungsi rekursif di mana pemanggilan rekursif merupakan pernyataan terakhir yang dijalankan oleh fungsi. Setelah pemanggilan rekursif dilakukan, tidak ada lagi operasi lain yang perlu dijalankan. Fungsi *tail recursion* lebih baik dibandingkan dengan rekursi biasa (non-*recursion*) karena dapat dioptimalkan oleh compiler. Optimisasi ini cukup sederhana, karena pemanggilan rekursif merupakan pernyataan terakhir, maka tidak ada lagi proses yang harus dilakukan oleh fungsi saat ini. Oleh karena itu penyimpanan stack frame saat ini dapat dihindari dengan langsung mengembalikan kontrol ke awal fungsi menggunakan perulangan (loop) atau perintah seperti goto.

BAB III

LANGKAH LANGKAH PRAKTIKUM

3.1 Alat dan Bahan

1. *GitBash.*
2. *Visual Studio Code.*
3. Modul praktikum.

3.2 Pembuatan *File* dan *Folder* Menggunakan *GitBash*

```
syafn@Dandika MINGW64 ~
$ cd /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma

syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (ma
in)
$ mkdir praktikum-3.

syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (ma
in)
$ cd praktikum-3.

syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma/pra
ktikum-3. (main)
$ npm init -y
Wrote to C:\prak.sdt_alg\2025573010039_strukturdatadanalgoritma\praktikum-3.\pac
kage.json:

{
  "name": "praktikum-3.",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Gambar 1.0 Membuat *Folder* untuk Praktikum Hari ini

```
syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma/pra
ktikum-3. (main)
$ touch 01-function-dasar.js

syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma/pra
ktikum-3. (main)
$ touch 02-arrow-callback.js

syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma/pra
ktikum-3. (main)
$ touch 03-array-dasar.js

syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma/pra
ktikum-3. (main)
$ touch 04-array-method.js

syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma/pra
ktikum-3. (main)
$ touch 05-rekursi.js
```

Gambar 1.1 Membuat *File* dengan *GitBash* Buat Praktikum

3.2 Percobaan dan Latihan

3.2.1 *Function* Dasar.

```
//1. function tanpa parameter dan return
function sapa() {
    console.log('Halo! selamat datang di praktikum JavaScript');
}
sapa();
sapa();


//2. dengan parameter

function sapaNama(nama) {
    console.log(`Halo, ${nama}! Selamat belajar.`);
}
sapaNama('Budi');
sapaNama('Siti');

// beberapa parameter dan return value

function tambah(angka1, angka2) {
        const hasil = angka1 + angka2;
        return hasil;
}

const hasilPenjumlahan = tambah(10,25);
console.log('10 + 25 =', hasilPenjumlahan);
console.log('7 + 13 =', tambah(7,13));

function hitung(nilai, pengali = 2){
    return nilai * pengali;
}
console.log(hitung(5));
console.log(hitung(5,3));

const pesanGlobal = 'Saya ada di mana saja';

function cekScope(){
    const pesanLokal = 'Saya hanya ada di dalam function ini';
    console.log(pesanGlobal);
    console.log(pesanLokal);
}

cekScope();
console.log(pesanGlobal);
```

Gambar 3.2.1.0 Kodingan *Function* Dasar

```
Halo! selamat datang di praktikum JavaScript
Halo! selamat datang di praktikum JavaScript
Halo, Budi! Selamat belajar.
Halo, Siti! Selamat belajar.
10 + 25 = 35
7 + 13 = 20
10
15
Saya ada di mana saja
Saya hanya ada di dalam function ini
Saya ada di mana saja
```

Gambar 3.2.1.1 Hasil Koding *Function* Dasar

```
// latihan 1
console.log('latihan 1');

function kurang(a,b){
    const hasil = a - b;
    return hasil;
}

function kali(a,b){
    const hasil = a * b;
    return hasil;
}

function bagi(a,b){
     if(b == 0){
    console.log('error: tidak bisa membagi dengan noll');
    return null;
}
 return a / b;
}


const hasilPengurangan = kurang(6,3);
console.log('6 - 3 =', hasilPengurangan);

const hasilPengalian = kali(6,3);
console.log('6 * 3 =', hasilPengalian);

const hasilPembagian = bagi(6,3);
console.log('6/3 = ', hasilPembagian);

console.log('KALKULATOR')

function kalkulator(a, operator, b){
    let hasil;

    switch (operator){
        case '+':
            hasil = tambah(a,b);
            break;
        case '-':
            hasil = kurang(a,b);
            break;
```

```
        case '*':
            hasil = kali(a,b);
            break;
        case '/':
            hasil = bagi(a,b);
            break;
        default:
            return "operator tidak valid";

    }
    return hasil;

}

console.log(kalkulator(6,'+',8));
console.log(kalkulator(5,'-',5));
console.log(kalkulator(5,'*',5));
console.log(kalkulator(6,'/',3));
console.log(kalkulator(9,'/',0));
```

Gambar 3.2.1.2 Latihan *Function* Dasar

```
latihan 1
6 - 3 = 3
6 * 3 = 18
6/3 =  2
KALKULATOR
14
0
25
2
error: tidak bisa membagi dengan noll
```

Gambar 3.2.1.3 Hasil Latihan *Function* Dasar

3.2.2 *Arrow Function* dan *Callback.*

```
function kuadratBiasa(x){
    return x * x;
}

const kuadrat = (x) =>{
    return x * x;
};

const kuadratRingkas = x => x * x;

console.log('=== perbandingan penulisan====');
console.log('biasa : ', kuadratBiasa(5));
console.log('arrow : ', kuadrat(5));
console.log('ringkas : ', kuadratRingkas(5));

const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama,waktu) => `selamat ${waktu}, ${nama}`;

console.log('luas 4 x 6 : ', luas(4,6));
console.log(salam('budi','pagi'));

function lakukanOperasi(angka, operasiCallback){
    console.log(`angka awal: ${angka}`);
    const hasil = operasiCallback(angka);
    console.log(`hasil setelah operasi : ${hasil}`);
}
```

```
console.log('callback');

lakukanOperasi(7,kuadratRingkas);
lakukanOperasi(18, x => x + 100);
lakukanOperasi(20, function(x){
    return x / 2;
});

console.log('setTimeout (callback)');
console.log('pesan 1: sebelum timer');

setTimeout(() => {
 console.log('Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)');
}, 1000);

console.log('Pesan 2: Setelah mendaftarkan timer');
```

Gambar 3.2.2.0 Kodingan *Arrow Function* dan *Callback*

```
=== perbandingan penulisan====
biasa :  25
arrow :  25
ringkas :  25
luas 4 x 6 :  24
selamat pagi, budi
callback
angka awal: 7
hasil setelah operasi : 49
angka awal: 18
hasil setelah operasi : 118
angka awal: 20
hasil setelah operasi : 10
setTimeout (callback)
pesan 1: sebelum timer
Pesan 2: Setelah mendaftarkan timer
```

Gambar 3.2.2.1 Hasil Koding

```
console.log('Latihan 2');

const keHurufBesar = (teks) => teks.toUpperCase();
const tambahSeru = (teks) => teks + '!';
const hitungKata = (teks) => teks.split(' ').length;

function prosesKalimat(kalimat, transformasiCallback){
    const hasil = transformasiCallback(kalimat);
    return hasil;
}
console.log(prosesKalimat('belajar javascript itu menyenangkan', tambahSeru));
console.log(prosesKalimat('belajar javascript itu menyenangkan', keHurufBesar));
console.log(prosesKalimat('belajar javascript itu menyenangkan', hitungKata));
```

Gambar 3.2.2.3 Latihan Koding *Arrow* dan *Callback*

```
Latihan 2
belajar javascript itu menyenangkan!
BELAJAR JAVASCRIPT ITU MENYENANGKAN
4
```

Gambar 3.2.2.4 Hasil Latihan Koding

3.2.3 *Array* Dasar.

```
const mahasiswa = ['albudi', 'phainon', 'aventurine', 'kiana'];
const nilai = [9,87,89, 30];

console.log('array awal');
console.log('mahasiswa: ', mahasiswa);
console.log('nilai : ', nilai);
console.log('jumlah mahasiswa: ', mahasiswa.length);

console.log('akses elemen');
console.log('elemen pertama : ', mahasiswa[0]);
console.log('elemen ketiga : ', mahasiswa[2]);
console.log('elemen terakhir : ', mahasiswa[mahasiswa.length - 1]);

mahasiswa[1] = 'siti rahayu';
console.log('setelah push :', mahasiswa);

console.log('manipulasi array');
mahasiswa.push('Aang');
console.log('setelah push : ',mahasiswa);

mahasiswa.unshift('Zahra');
console.log('setelah unshift : ',mahasiswa);

const dihapusAkhir = mahasiswa.pop();
console.log('setelah pop : ',mahasiswa);

const dihapusAwal = mahasiswa.shift();
console.log('setelah shift : ',mahasiswa);

console.log('pencarian');
console.log('indeks phainon :', mahasiswa.indexOf('phainon'));
console.log('ada albudi? : ', mahasiswa.includes ('albudi'));
console.log('ada kiana? : ', mahasiswa.includes('kiana'));

const sebagian = nilai.slice(1, 4);
console.log('nilai asli :'. nilai);
console.log('slice [1,4] : ', sebagian);
```

Gambar 3.2.3.0 Koding *Array* Dasar

```
array awal
mahasiswa:  [ 'albudi', 'phainon', 'aventurine', 'kiana' ]
nilai :  [ 9, 87, 89, 30 ]
jumlah mahasiswa:  4
akses elemen
elemen pertama :  albudi
elemen ketiga :  aventurine
elemen terakhir :  kiana
setelah push : [ 'albudi', 'siti rahayu', 'aventurine', 'kiana' ]
manipulasi array
setelah push :  [ 'albudi', 'siti rahayu', 'aventurine', 'kiana', 'Aang' ]
setelah unshift :  [ 'Zahra', 'albudi', 'siti rahayu', 'aventurine', 'kiana', 'Aang' ]
setelah pop :  [ 'Zahra', 'albudi', 'siti rahayu', 'aventurine', 'kiana' ]
setelah shift :  [ 'albudi', 'siti rahayu', 'aventurine', 'kiana' ]
pencarian
indeks phainon : -1
ada albudi? :  true
ada kiana? :  true
nilai asli : [ 9, 87, 89, 30 ]
slice [1,4] :  [ 87, 89, 30 ]
```

Gambar 3.2.3.1 Hasil Koding

```
console.log('latihan 3');

const daftarBelanja = ['beras','sosis','ayam','daging','ketela'];

console.log('daftar berlanja');
for(let i = 0; i < daftarBelanja.length; i++){
    console.log(`${i + 1}. ${daftarBelanja[i]}`);
}

daftarBelanja.push('sayur','tahu');
console.log('setelah di push : ', daftarBelanja);

const diHapus = daftarBelanja.shift();
console.log('item yang dihapus :', diHapus);
console.log ('setelah di shift : ', daftarBelanja);

console.log('ada susu? :', daftarBelanja.includes('susu'));

console.log('elemen dalam daftar belanja terakhir : ', daftarBelanja[daftarBelanja.length -1]);
```

Gambar 3.2.3.2 Latihan Koding *Array* Dasar

```
daftar berlanja
1. beras
2. sosis
3. ayam
4. daging
5. ketela
setelah di push :  [
  'beras',  'sosis',
  'ayam',   'daging',
  'ketela', 'sayur',
  'tahu'
]
item yang dihapus : beras
setelah di shift :  [ 'sosis', 'ayam', 'daging', 'ketela', 'sayur', 'tahu' ]
ada susu? : false
elemen dalam daftar belanja terakhir :  tahu
```

Gambar 3.2.3.3 Hasil Latihan Koding

3.2.4 *Array Method*.

```
const nilaiMahasiswa = [75,90,55,82,68,95,48, 77];

console.log('forEach, menampilkan semua nilai');
nilaiMahasiswa.forEach((nilai,indeks) => {
    console.log(`Mahasiswa-${indeks = 1}: ${nilai}`);

});

const gradeHuruf = nilaiMahasiswa.map(nilai => {
    if (nilai >= 90) return 'A';
    if (nilai >= 80) return 'B';
    if (nilai >= 70) return 'C';
    if (nilai >= 60) return 'D';
    return 'E';
});

console.log('map, nilai ke grade');
console.log('nilai : ', nilaiMahasiswa);
console.log('grade :', gradeHuruf);

const nilaiLulus = nilaiMahasiswa.filter(nilai => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter(nilai => nilai < 60);
```

```
const totalNilai = nilaiMahasiswa.reduce((akumulator,nilai) =>{
    return akumulator + nilai;
},0);
const rataRata = totalNilai / nilaiMahasiswa.length;

console.log('reduce statistik nilai');
console.log('total nilai :', totalNilai);
console.log('Rata-rata :', rataRata.toFixed(2));

const rataRataNilaiLulus = nilaiMahasiswa
    .filter(nilai => nilai >= 60)
    .reduce((acc, val, idx, arr) => {
        return acc + val / arr.length;
},0);

console.log('method chaining');
console.log('rata-rata nilai lulus : ', rataRataNilaiLulus.toFixed(2));
```

Gambar 3.2.4.1 Koding *Array Method*

```
forEach, menampilkan semua nilai
Mahasiswa-1: 75
Mahasiswa-1: 90
Mahasiswa-1: 55
Mahasiswa-1: 82
Mahasiswa-1: 68
Mahasiswa-1: 95
Mahasiswa-1: 48
Mahasiswa-1: 77
map, nilai ke grade
nilai :  [
  75, 90, 55, 82,
  68, 95, 48, 77
]
grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]
filter : lulus dan tidak
semua nilai :  [
  75, 90, 55, 82,
  68, 95, 48, 77
]
```

```
filter : lulus dan tidak
semua nilai :  [
  75, 90, 55, 82,
  68, 95, 48, 77
]
nilai lulus :  [ 75, 90, 82, 68, 95, 77 ]
nilai gagal :  [ 55, 48 ]
reduce statistik nilai
total nilai : 590
Rata-rata : 73.75
method chaining
rata-rata nilai lulus :  81.17
```

Gambar 3.2.4.2 Hasil Koding

```
console.log('latihan 4'); //==========================================================================

const produk = [
    { nama: 'Laptop', harga: 8500000, stok: 5 },
    { nama: 'Mouse', harga: 150000, stok: 0 },
    { nama: 'Keyboard', harga: 450000, stok: 12 },
    { nama: 'Monitor', harga: 3200000, stok: 0 },
    { nama: 'Headset', harga: 350000, stok: 8 }
];

const produkTersedia = produk.filter(p => p.stok > 0);
console.log("produk tersedia:", produkTersedia);

const namaProdukTersedia = produkTersedia.map(p => p.nama);
console.log("daftar nama produk yang ada:", namaProdukTersedia);

const totalInventaris = produkTersedia.reduce((total, p) => {
    return total + (p.harga * p.stok);
}, 0);
console.log("total nilai inventaris (Tersedia): Rp" + totalInventaris.toLocaleString('id-ID'));

console.log("status");
produk.forEach(p => {
    const status = p.stok > 0 ? "[TERSEDIA]" : "[HABIS]";
    const formatHarga = "Rp" + p.harga.toLocaleString('id-ID');

    if (p.stok > 0) {
        console.log(`${status} ${p.nama} - ${formatHarga} (${p.stok} unit)`);
    } else {
        console.log(`${status} ${p.nama} - ${formatHarga}`);
    }
});
```

Gambar 3.2.4.3 Latihan Koding *Array Method*

```
latihan 4
produk tersedia: [
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]
daftar nama produk yang ada: [ 'Laptop', 'Keyboard', 'Headset' ]
total nilai inventaris (Tersedia): Rp50.700.000
status
[TERSEDIA] Laptop - Rp8.500.000 (5 unit)
[HABIS] Mouse - Rp150.000
[TERSEDIA] Keyboard - Rp450.000 (12 unit)
[HABIS] Monitor - Rp3.200.000
[TERSEDIA] Headset - Rp350.000 (8 unit)
```

Gambar 3.2.4.4 Hasil Koding

3.2.5 Rekursi.

```
function faktorial(n){
    if (n<= 1) return 1;

    return n * faktorial(n-1);
}

console.log('faktorial');
console.log('0! =', faktorial(0));
console.log('1! =', faktorial(1));
console.log('5! =', faktorial(5));
console.log('7! =', faktorial(7));

function fibonacci(n){
    if (n === 0) return 0;
    if ( n === 1) return 1;

    return fibonacci (n-1) + fibonacci (n-2);
}
```

```
console.log('');

function jumlahArray (arr, indeks = 0){
    if (indeks == arr.length) return 0;

    return arr[indeks] + jumlahArray(arr,indeks + 1);
}

const angka = [3,7,2,9,5];
console.log('jumlah array dengn angka reukrsi');
console.log(' array :', angka);
console.log('jumalah : ', jumlahArray(angka));

function countdown(n){
    if (n < 0) {
        console.log('selesai');
        return;
    }
    console.log(n);
    countdown(n-1);
}

console.log('countrdown rekursif');
countdown(5);
```

Gambar 3.2.5.1 Koding Rekursi

```
faktorial
0! = 1
1! = 1
5! = 120
7! = 5040

jumlah array dengn angka reukrsi
 array : [ 3, 7, 2, 9, 5 ]
jumalah :  26
countrdown rekursif
5
4
3
2
1
0
selesai
```

Gambar 3.2.5.2 Hasil Koding

3.3 Tugas

3.3.1 Sistem Nilai Mahasiswa

```javascript
const dataMahasiswa = [
    { nama: 'Albudi', nilai: 96 },
    { nama: 'Phainon', nilai: 88 },
    { nama: 'Stelle', nilai: 50 },
    { nama: 'Kiana', nilai: 90 },
    { nama: 'Caelus', nilai: 65 },
    { nama: 'Lyuuci', nilai: 90 }
];

function hitungStatistik(arrMahasiswa){

    let total = arrMahasiswa.length;
    let jumlah = 0;
    let tertinggi = arrMahasiswa[0].nilai;
    let terendah = arrMahasiswa[0].nilai;

    arrMahasiswa.forEach(function(m){
        jumlah += m.nilai;

        if(m.nilai > tertinggi){
            tertinggi = m.nilai;
        }

        if(m.nilai < terendah){
            terendah = m.nilai;
        }
```

```javascript
    return {
        total: total,
        rataRata: (jumlah / total).toFixed(2),
        tertinggi: tertinggi,
        terendah: terendah
    };

ction filterLulus(arrMahasiswa, batasLulus){
    return arrMahasiswa.filter(function(m){
        return m.nilai >= batasLulus;
    });

ction tambahGrade(arrMahasiswa){
    return arrMahasiswa.map(function(m){

        let grade;

        if(m.nilai >= 85){
            grade = 'A';
        } else if(m.nilai >= 75){
            grade = 'B';
        } else if(m.nilai >= 65){
            grade = 'C';
        } else if(m.nilai >= 50){
```

```javascript
        } else {
            grade = 'E';
        }

        return {
            nama: m.nama,
            nilai: m.nilai,
            grade: grade
        };

    });
}

const statistik = hitungStatistik(dataMahasiswa);
const lulus = filterLulus(dataMahasiswa, 75);
const denganGrade = tambahGrade(dataMahasiswa);

console.log("=== DATA MAHASISWA ===");
dataMahasiswa.forEach(function(m, i){
    console.log((i+1) + ". " + m.nama + " - " + m.nilai);
});

console.log("\n=== STATISTIK ===");
console.log("Total Mahasiswa :", statistik.total);
console.log("Rata-rata :", statistik.rataRata);
console.log("Tertinggi :", statistik.tertinggi);
console.log("Terendah :", statistik.terendah);


console.log("\n=== MAHASISWA LULUS ===");
lulus.forEach(function(m, i){
    console.log((i+1) + ". " + m.nama + " - " + m.nilai);
});


console.log("\n=== DATA DENGAN GRADE ===");
denganGrade.forEach(function(m, i){
    console.log((i+1) + ". " + m.nama + " - " + m.nilai + " (Grade " + m.grade + ")");
});
```

Gambar 3.3.1.1 Sistem Nilai Mahasiswa

```
=== DATA MAHASISWA ===
1. Albudi - 96
2. Phainon - 88
3. Stelle - 50
4. Kiana - 90
5. Caelus - 65
6. Lyuuci - 90

=== STATISTIK ===
Total Mahasiswa : 6
Rata-rata : 79.83
Tertinggi : 96
Terendah : 50
```

```
=== MAHASISWA LULUS ===
1. Albudi - 96
2. Phainon - 88
3. Kiana - 90
4. Lyuuci - 90

=== DATA DENGAN GRADE ===
1. Albudi - 96 (Grade A)
2. Phainon - 88 (Grade A)
3. Stelle - 50 (Grade D)
4. Kiana - 90 (Grade A)
5. Caelus - 65 (Grade C)
6. Lyuuci - 90 (Grade A)
```

Gambar 3.3.1.2 Hasil Koding

3.3.2 Pangkat dan Palindrom Rekursif

```javascript
function pangkat(basis, eksponen) {
    if (eksponen === 0) {
        return 1;
    }
    return basis * pangkat(basis, eksponen - 1);
}

console.log("Pangkat:");
console.log(pangkat(2, 3));
console.log(pangkat(5, 2));
console.log(pangkat(3, 4));

function balikString(str) {
    if (str.length <= 1) {
        return str;
    }
    return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
}

console.log("\nBalik String:");
console.log(balikString("halo"));
console.log(balikString("javascript"));

function cekPalindrom(str) {
    return str === balikString(str);
}

console.log("\nCek Palindrom:");
console.log("katak:", cekPalindrom("katak"));
console.log("civic:", cekPalindrom("civic"));
console.log("level:", cekPalindrom("level"));
console.log("halo:", cekPalindrom("halo"));
```

Gambar 3.3.2.1 Tugas Pangkat dan Palindrom Rekursif

```
Pangkat:
8
25
81

Balik String:
olah
tpircsavaj

Cek Palindrom:
katak: true
civic: true
level: true
halo: false
```

Gambar 3.3.2.2 Hasil Koding

3.4 Mengirim Ke *GitHub*

```
syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (
in)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        praktikum-3/

nothing added to commit but untracked files present (use "git add" to track)
```

Gambar 3.4.1 Melihat Status *Git*

```
syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (ma
in)
$ git add praktikum-3/
warning: in the working copy of 'praktikum-3/package.json', LF will be replaced
by CRLF the next time Git touches it
```

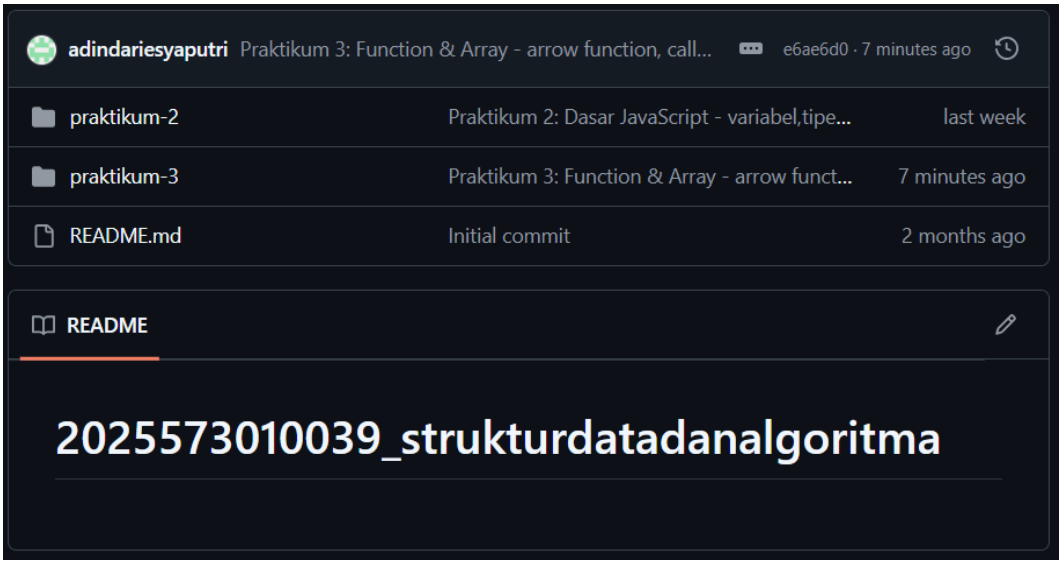
Gambar 3.4.2 Menambahkan Praktikum-3 ke dalam *Git*

```
syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (main)
$ git commit -m "Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi"
[main e6ae6d0] Praktikum 3: Function & Array - arrow function, callback, iterasi array, rekursi
 8 files changed, 508 insertions(+)
 create mode 100644 praktikum-3/01-function-dasar.js
 create mode 100644 praktikum-3/02-arrow-callback.js
 create mode 100644 praktikum-3/03-array-dasar.js
 create mode 100644 praktikum-3/04-array-method.js
 create mode 100644 praktikum-3/05-rekursi.js
 create mode 100644 praktikum-3/package.json
 create mode 100644 praktikum-3/tugas/tugas-1.js
 create mode 100644 praktikum-3/tugas/tugas-2.js
```

Gambar 3.4.3 Membuat *Folder* Untuk Mengumpulkan *File-File* Yang di Perlukan

```
syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 4.94 KiB | 1.65 MiB/s, done.
Total 12 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/adindariesyaputri/2025573010039_strukturdatadanalgoritma.git
   f9b80b4..e6ae6d0  main -> main
```

Gambar 3.4.4 *Push Folder* ke *GitHub*



Gambar 3.4.5 *Folder* Praktikum-3 Berhasil di-Unggah ke *GitHub*

BAB IV

PEMBAHASAN DAN ANALISA

1. Alat dan Bahan

Visual studio code merupakan sebuah .*GitBash* merupakan antarmuka baris perintah yang mirip dengan unix untuk *Microsoft Windows* yang memungkinkan *user* menjalankan perintah *git* dan mengelola repositori dari terminal

2. Pembuatan *File* dan *Folder*

Untuk membuat *file* dimulai dengan membuka *git* Bash lalu mengetik cd-nama repository-mu untuk masuk ke dalam *folder* tersebut. Lalu ketik mkdir praktikum-3, cd praktikum-3 untuk masuk ke dalam *folder* tersebut lalu download Node.js dengan cara npm init -y lalu konfirmasi kan dengan ls. Setelah selesai kita buat *file* dengan menggunakan command touch-nama *file*.js. Dan membuat *folder* lain dan *file* untuk tugas dengan cara yang sama seperti sebelumnya.

3. Percobaan dan Latihan

Dalam percobaan kali ini kita akan mencoba *coding function* dasar, *arrow function dan Callback*, *array* dasar, *array method*, dan rekursi. Terdapat juga beberapa latihan yang diberikan.

- *Function* Dasar.

Function dasar digunaka untuk mengelompokkan kode agar lebih rapi, mudah dibaca dan dapat digunakan kembali. Pertama ada *function* tanpa parameter dan *return value*.

```
function sapa() {

    console.log('Halo! selamat datang di praktikum JavaScript');

}

sapa();

sapa();
```

Pada *function* sapa() dibuat agar tidak memiliki parameter dan juga *return value*. Pesan ini hanya menampilkan pesan yang menggunakan *console.log*(). *Function* tanpa parameter biasa nya digunakan untuk menjalankan perintah sederhana tanpa input.

- Selanjutnya *function* dengan parameter

```
function sapaNama(nama) {

    console.log(`Halo, ${nama}! Selamat belajar.`);

}

sapaNama('Budi');

sapaNama('Siti');
```

Function sapaNama(nama) memiliki parameter nama yang digunakan untuk menerima nilai dari luar *function*. Nilai tersebut ditampilkan dalam bentuk balasan pesan, *function* dipanggil dengan nilai Budi dan Siti sehingga menghasilkan output yang berbeda. *Function* dengan parameter membuat *function* lebih fleksibel dan dapat digunakan kembali dengan nilai yang berbeda.

- *Function* dengan *return value*

```
function tambah(angka1, angka2) {

        const hasil = angka1 + angka2;

        return hasil;

}
```

Function tambah() menerima dua parameter yaitu angka1 dan angka2. *Function* ini menjumlahkan kedua nilai tersebut kemudian mengembalikan hasilnya menggunakan *return*. *Return* digunakan agar dapat di simpan dan di pakai kembali oleh program.

- *Function* dengan *default* parameter

```
function hitung(nilai, pengali = 2){

    return nilai * pengali;

}
```

Function hitung() menggunakan *default* parameter pada pengali dengan nilai 2. Jika parameter kedua tidak diisi makan *function* akan menggunakan nilai *default*. *Function* ini mempermudah penggunaan *function* karena tidak perlu selalu mengisi semua parameter yang ada.

- *Function scope* variabel

```
const pesanGlobal = 'Saya ada di mana saja';



function cekScope(){

    const pesanLokal = 'Saya hanya ada di dalam function ini';

    console.log(pesanGlobal);

    console.log(pesanLokal);
```

Pada bagian ini terdapat variabel *global* pesan*Global* yang dapat diakses di seluruh program. Sedangkan variabel PesanLokal hanya bisa digunakan di dalam *function* cek*Scope*(). Penggunaan *scope* variabel penting untuk menghindari konflik antar variabel dalam program.

➢ *Arrow Function* dan *Callback*

- Perbandingan *function* biasa dengan *arrow function*.

```
function kuadratBiasa(x){

    return x * x;

}

const kuadrat = (x) =>{

    return x * x;

};

const kuadratRingkas = x => x * x;
```

Dapat kita lihat pada bagian ini terdapat tiga cara membuat *function* untuk menghitung kuadrat suatu angka, *Function* biasa, *arrow function* standar, dan *arrow function* ringkas tanpa parameter dan *return value*. Ketiga *function* tersebut memiliki fungsi yang sama yaitu menghitung nilai kuadrat dan ketiga cara tersebut menghasilkan output yang sama, hanya berbeda pada cara penulisan.

- *Arrow function* dengan beberapa parameter

```
const luas = (panjang, lebar) => panjang * lebar;

const salam = (nama,waktu) => `selamat ${waktu}, ${nama}`;
```

Di bagian ini dibuat dua *arrow function*, yaitu untuk menghitung luas dan juga untuk menampilkan pesan salam. Dengan menggunakan *arrow function*, penulisan *function* bisa dilakukan dengan mudah.

- *Callback function*

```
function lakukanOperasi(angka, operasiCallback){

    console.log(`angka awal: ${angka}`);

    const hasil = operasiCallback(angka);

    console.log(`hasil setelah operasi : ${hasil}`);

}
```

Function Callback adalah *function* yang dikirim sebagai parameter ke *function* lain. *Function* ini menerima dua parameter, yaitu angka sebagai nilai awal dan operasi*Callback* sebagai *function* yang akan dijalankan. *Callback* membuat *function* menjadi lebih fleksibel digunakan.

- *setTimeout* and *Callback*

```
console.log('setTimeout (Callback)');

console.log('pesan 1: sebelum timer');



setTimeout(() => {

 console.log('Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)');

}, 1000);
```

Function kali ini menggunakan *setTimeout* yang merupakan *function* bawaan JavaScript untuk menjalankan kode setelah waktu tertentu. seTimeout() menerima *Callback function*, 1000 berarti 1000 milidetik atau 1 detik. Walaupun pesan 3 ditulis lebih dulu dalam kode, tetapi dijalankan terakhir karena menggunakan timer.

➢ *Array* Dasar

- Membuat *array*

```
const mahasiswa = ['albudi', 'phainon', 'aventurine', 'kiana'];

const nilai = [9,87,89, 30];
```

Disini terdapat dua buah *array*, *array* pertama bernama mahasiswa dengan indeks albudi, phainon,aventurine dan kiana. Sedangkan *array* kedua memiliki nama nilai dengan indeks 9,87,89 dan 30.

```
console.log('array awal');

console.log('mahasiswa: ', mahasiswa);

console.log('nilai : ', nilai);

console.log('jumlah mahasiswa: ', mahasiswa.length);
```

Console.log digunakan untuk menampilkan isi *array*, sedangkan .length digunakan untuk mengetahui jumlah data yang ada dalam *array*.

- Mengakses elemen di *array*

```
console.log('akses elemen');

console.log('elemen pertama : ', mahasiswa[0]);

console.log('elemen ketiga : ', mahasiswa[2]);

console.log('elemen terakhir : ', mahasiswa[mahasiswa.length - 1]);
```

Array dimulai dari indeks 0, maka mahasiswa[0] artinya elemen pertama yaitu albudi, kalau mahasiswa[2] artinya elemen ketiga yaitu nilai aventurine. Untuk mengetahui elemen terakhir kita hanya perlu mengetik [mahasiswa.length-1], jumlah semua mahasiswa dikurangi satu.

- Mengubah elemen di *array*

```
mahasiswa[1] = 'siti rahayu';

console.log('setelah push :', mahasiswa);
```

Data pada indeks pertama (saat ini phainon) akan diubah menjadi “siti rahayu”, ini salah satu cara yang digunakan untuk mengganti nilai dalam *array*.

- Memanipulasi *array*
  - *Push*, menambah data di bagian akhir

```
mahasiswa.push('Aang');

console.log('setelah push : ',mahasiswa);
```

  - *Unshift*, menambah data di bagian awal

```
mahasiswa.unshift('Zahra');

console.log('setelah unshift : ',mahasiswa);
```

  - *Pop*, menghapus data di bagian akhir

```
const dihapusAkhir = mahasiswa.pop();

console.log('setelah pop : ',mahasiswa);
```

Nilai yang dihapus disimpan dalam variabel dihapusAkhir.

  - *Shift*, menghapus data di bagian awal

```
const dihapusAwal = mahasiswa.shift();

console.log('setelah shift : ',mahasiswa);
```

Nilai yang dihapus disimpan dalam variabel dihapusAwal

- Pencarian data dalam *array*

```
console.log('pencarian');

console.log('indeks phainon :', mahasiswa.indexOf('phainon'));

console.log('ada albudi? : ', mahasiswa.includes ('albudi'));

console.log('ada kiana? : ', mahasiswa.includes('kiana'));
```

indexof() akan mencari posisi data yang kita cari dalam *array*, kalau includes() akan mengecek apakah data ada dalam *array*. Jika ada maka akan ditampilkan true, jika tidak maka akan ditampilkan false.

- Mengambil sebagian *array*

```
const sebagian = nilai.slice(1, 4);

console.log('nilai asli :', nilai);

console.log('slice [1,4] : ', sebagian);
```

slice (1,4) bermaksud untuk mengambil data dari indeks 1 sampai sebelum indeks 4, *array* asli nya tidak akan berubah dan data hasil akan disimpan dalam variabel sebagian.

➢ *Array Method*

○ forEach(), menampilkan semua nilai

```
const nilaiMahasiswa = [75,90,55,82,68,95,48, 77];



console.log('forEach, menampilkan semua nilai');

nilaiMahasiswa.forEach((nilai,indeks) => {

    console.log(`Mahasiswa-${indeks = 1}: ${nilai}`);

});
```

forEach() digunakan untuk melakukan perulangan pada *array* dengan maksud parameter nilai yang di maksud sebagai nilai dalam setiap elemen dan indeks untuk posisi elemen dalam *array* nya. Setiap nilai ditampilkan menggunakan *console.log*. forEach biasa nya juga digunakan untuk menampilkan atau memproses data tanpa membuat *array* yang baru.

- *map()*, mengubah nilai menjadi grade

```
const gradeHuruf = nilaiMahasiswa.map(nilai => {
    if (nilai >= 90) return 'A';
    if (nilai >= 80) return 'B';
    if (nilai >= 70) return 'C';
    if (nilai >= 60) return 'D';
    return 'E';
});
```

Map digunakan untuk mengubah isi *array*, *method*e ini menghasilkan *array* baru dan nilai angka akan diubah menjadi nilai huruf.

- *filter()*,penyaringan data

```
const nilaiLulus = nilaiMahasiswa.filter(nilai => nilai >= 60);

const nilaiGagal = nilaiMahasiswa.filter(nilai => nilai < 60);
```

filter() digunakan untuk menyaring data berdasarkan kondisi dan dapat menghasilkan *array* yang baru, *filter* juga sangat berguna untuk memisahkan data sesuai kriteria tertentu.

- *reduce()*,menghitung total dan rata-rata

```
const totalNilai = nilaiMahasiswa.reduce((akumulator,nilai) =>{

    return akumulator + nilai;

},0);

const rataRata = totalNilai / nilaiMahasiswa.length;
```

reduce() digunakan untuk menggabungkan semua nilai menjadi satu nilai, akumulator akan menyimpan hasil sementara. 0 adalah nilai awal nya, reduce biasa nya digunakan untuk menghitung total, mencari nilai maksimum dan minimum.

- *Method* chaining

```
const rataRataNilaiLulus = nilaiMahasiswa

    .filter(nilai => nilai >= 60)

    .reduce((acc, val, idx, arr) => {

        return acc + val / arr.length;

},0);
```

Method chaining, merupakan penggabungan beberapa *method* sekaligus. Yang digabungkan di code ini adalah *filter*, untuk mengambil nilai lulus, dan reduce untuk menghitung rata-rata nya. *Method* chaining membuat kode lebih singkat dan juga efisien.

- Rekursi
  - Rekursi faktorial

```
function faktorial(n){

    if (n<= 1) return 1;



    return n * faktorial(n-1);

}
```

Function faktorial digunakan untuk menghitung nilai faktorial dari suatu bilangan.

```
if (n<= 1) return 1;
```

Ini merupakan kondisi berhenti nya sebuah *function* (base case)

```
return n * faktorial(n-1);
```

Ini adalah rekursi nya.

  - Rekursi fibonacci

```
function fibonacci(n){

    if (n === 0) return 0;

    if ( n === 1) return 1;



    return fibonacci (n-1) + fibonacci (n-2);

}
```

Fibonacci adalah deret angka di mana setiap angka merupakan jumlah dari dua angka sebelum nya.

```
if (n === 0) return 0;
```

Fungsi ini merupakan kondisi awal, yang dimana jika n adalah 0 maka fungsi langsung mengembalikkan nila 0.

```
if ( n === 1) return 1;
```

Fungsi ini sama seperti yang pertama, fungsi yang merupakan kondisi kedua, yang dimana jika n adalah 1 maka fungsi langsung mengembalikkan nila 1.

```
return fibonacci (n-1) + fibonacci (n-2);
```

Return ini merupakan rekursi nya. *Function* rekursi ini memanggil dirinya dua kali untuk menghitung nilai sebelum nya.

- Rekursi penjumlahan *array*

```
function jumlahArray (arr, indeks = 0){

    if (indeks == arr.length) return 0;



    return arr[indeks] + jumlahArray(arr,indeks + 1);

}
```

Function jumlah*Array*() digunakan untuk menjumlahkan seluruh elemen dalam *array* menggunakan rekursi.

```
indeks = 0
```

Merupakan nilai awal pada indeks.

```
 if (indeks == arr.length)
```

Fungsi yang digunakan untuk kondisi berhenti.

```
return arr[indeks] + jumlahArray(arr,indeks + 1);
```

Dan fungsi ini adalah rekursi nya.

- Rekursi countdown

```
function countdown(n){
  if (n < 0) {
    console.log('selesai');
    return;
  }
  console.log(n);
  countdown(n-1);
}
```

Function countdown() digunakan untuk menghitung mundur menggunakan rekursi. Jika n < 0 maka program nya akan berhenti, namun jika tidak angka ditampilkan lalu *function* nya akan dipanggil kembali

4. Latihan dan Tugas Bahasa JavaScript

   1. Sistem Nilai Mahasiswa

      Sistem nilai mahasiswa ini menggunakan *array* of object dan beberapa *function* seperti forEach, *filter* dan juga map. Program ini menghitung statistik nilai, menampilkan mahasiswa yang lulus, serta menambahkan grade pada setiap mahasiswa.

```
const dataMahasiswa = [
  { nama: 'Albudi', nilai: 96 },
  { nama: 'Phainon', nilai: 88 },
  { nama: 'Stelle', nilai: 50 },
  { nama: 'Kiana', nilai: 90 },
  { nama: 'Caelus', nilai: 65 },
```

```
  { nama: 'Lyuuci', nilai: 90 }

];
```

Merupakan *array* of object, dimana setiap object memiliki nama dan nilai di dalam nya.

```
function hitungStatistik(arrMahasiswa){
```

Function ini digunakan untuk menghitung total mahasiswa, rata-rata nilai,nilai tertinggi dan juga terendah.

```
let total = arrMahasiswa.length;

  let jumlah = 0;

  let tertinggi = arrMahasiswa[0].nilai;

  let terendah = arrMahasiswa[0].nilai;
```

Ini adalah fungsi yang digunakan untuk mencari hal tersebut.

```
arrMahasiswa.forEach(function(m){
```

Digunakan untuk membaca setiap data mahasiswa dimulai dari menjumlahkan semua nilai, mencari nilai tertinggi, dan mencari nilai yang terendah seperti yang bisa dilihat pada coding *function* dibawah ini.

```
jumlah += m.nilai;



    if(m.nilai > tertinggi){

        tertinggi = m.nilai;

    }



    if(m.nilai < terendah){

        terendah = m.nilai;
```

```
        }

    });
```

Untuk mengembalikkan hasil dalam bentuk object, kita menggunakan *function return*

```
return {

        total: total,

        rataRata: (jumlah / total).toFixed(2),

        tertinggi: tertinggi,

        terendah: terendah

    };
```

Untuk menampilkan mahasiswa yang lulus, kita menggunakan *function filter*Lulus, jika nilai lebih dari sama dengan batas lulus, maka mahasiswa akan dianggap lulus.

```
function filterLulus(arrMahasiswa, batasLulus){

    return arrMahasiswa.filter(function(m){

        return m.nilai >= batasLulus;

    });

}
```

Menggunakan *function* tambahGrade() untuk menambahkan grade huruf berdasarkan nilai, memakai *function* if untuk menentukan grade nya dan memakai *return* untuk mengembalikkan *array* baru dengan tambahan grade.

```
function tambahGrade(arrMahasiswa){
  return arrMahasiswa.map(function(m){

    let grade;

    if(m.nilai >= 85){
      grade = 'A';
    } else if(m.nilai >= 75){
      grade = 'B';
    } else if(m.nilai >= 65){
      grade = 'C';
    } else if(m.nilai >= 50){
      grade = 'D';
    } else {
      grade = 'E';
    }

    return {
      nama: m.nama,
      nilai: m.nilai,
      grade: grade
```

```
        };


    });

}
```

Untuk menjalankan *function* nya kita menggunakan const untuk menunjukkan statistik,grade dan lulus. Sementara data mahasiswa kita menggunakan forEach untuk menampilkan nya

```
const statistik = hitungStatistik(dataMahasiswa);

const lulus = filterLulus(dataMahasiswa, 75);

const denganGrade = tambahGrade(dataMahasiswa);


console.log("=== DATA MAHASISWA ===");

dataMahasiswa.forEach(function(m, i){

    console.log((i+1) + ". " + m.nama + " - " + m.nilai);

});


console.log("\n=== STATISTIK ===");

console.log("Total Mahasiswa :", statistik.total);

console.log("Rata-rata :", statistik.rataRata);

console.log("Tertinggi :", statistik.tertinggi);

console.log("Terendah :", statistik.terendah);



console.log("\n=== MAHASISWA LULUS ===");

lulus.forEach(function(m, i){
```

```
    console.log((i+1) + ". " + m.nama + " - " + m.nilai);
});



console.log("\n=== DATA DENGAN GRADE ===");
denganGrade.forEach(function(m, i){
    console.log((i+1) + ". " + m.nama + " - " + m.nilai + " (Grade " + m.grade + ")");
});
```

2. Pangkat dan Palindrom Rekursif.

   Menggunakan *function* pangkat() untuk menghitung nilai pangkat menggunakan rekursi.

```
function pangkat(basis, eksponen) {
    if (eksponen === 0) {
        return 1;
    }
    return basis * pangkat(basis, eksponen - 1);
}
```

   Kondisi berhenti atau base case terletak pada *function* if (eksponen === o) dan rekursi terdapat pada *function return* basis * pangkat(basis, eksponen - 1).

```
function balikString(str) {
    if (str.length <= 1) {
        return str;
    }
    return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
}
```

Function balikString() digunakan untuk membalikkan teks menggunakan rekursi. Str. length <= 1 berfungsi untuk mejadi kondisi berhenti, str[str.length - 1] berfungsi untuk mengambil huruf terakhir dan str.slice() berfungsi untuk mengambil sisa string nya.

```
console.log("\nBalik String:");

console.log(balikString("halo"));

console.log(balikString("javascript"));
```

Selanjut nya kita memakai *function* cek palindrom() yang mana digunakan untuk mengecek apakah kata termasuk palindrom atau tidak. Palindrom sendiri adalah kata yang dibaca sama dari depan maupun belakang.

```
function cekPalindrom(str) {

    return str === balikString(str);

}



console.log("\nCek Palindrom:");

console.log("katak:", cekPalindrom("katak"));

console.log("civic:", cekPalindrom("civic"));

console.log("level:", cekPalindrom("level"));

console.log("halo:", cekPalindrom("halo"));
```

5. Upload Praktikum ke *GitHub*

Pengumploadan *folder* praktkium ke dalam *GitHub* sangat lah mudah. Pertama membuka *GitBash* lalu kembali ke root repository dengan mengetik cd… setelah itu kita mengetik *git* status yang berfungsi untuk mengecek *file* yang berubah atau belum di dalam commit.

git add praktikum-3/

Seterus nya kita menambahkan *folder* praktikum-3/ untuk memasukkan perubahan ke staging area sebelum commit.

git commit -m "Praktikum 3: *Function* & *Array* - *arrow function*, *Callback*, iterasi *array*, rekursi"

Git commit -m digunakan untuk menyimpan perubahan dengan pesan keterangan.

git push origin main

Digunakan untuk mengirim *file* ke *GitHub*

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *function* dan *array* merupakan bagian penting dalam pemrograman JavaScript yang membantu dalam pengolahan data secara lebih efisien dan terstruktur. *Function* digunakan untuk mengelompokkan kode agar lebih rapi, mudah dipahami, serta dapat digunakan kembali. Selain itu, penggunaan *arrow function* dan *Callback function* dapat membuat penulisan kode menjadi lebih sederhana dan fleksibel.

Array digunakan untuk menyimpan banyak data dalam satu variabel dan memudahkan dalam pengolahan data menggunakan berbagai *method* seperti forEach, map, *filter*, dan reduce. *Method* tersebut sangat membantu dalam melakukan perulangan, penyaringan data, serta pengolahan nilai secara efisien.

Rekursi juga merupakan teknik penting dalam pemrograman yang memungkinkan sebuah *function* memanggil dirinya sendiri untuk menyelesaikan masalah secara bertahap. Konsep rekursi dapat diterapkan dalam berbagai kasus seperti faktorial, fibonacci, penjumlahan *array*, dan pengecekan palindrom. Selain itu, pemahaman tentang call stack dan *tail recursion* membantu dalam memahami proses eksekusi program serta mengoptimalkan penggunaan memori.

5.2 Saran

Adapun saran sebagai berikut:

1. Diperlukan ketelitian dalam menulis kode program untuk menghindari kesalahan sintaks maupun logika.
2. Memberikan waktu untuk melakukan latihan yang ada dalam modul agar mahasiswa bisa melakukan nya dengan tenang dan dapat menyerap apa yang di lakukan serta dapat membaca apa yang ada dalam modul tersebut.
3. Diperlukan pemahaman yang lebih mendalam mengenai error yang sering terjadi saat menggunakan rekursi agar dapat menghindari masalah seperti stack overflow.

DAFTAR PUSTAKA

Amanda Alodyasari. 2024. *Pengertian Rekursi, Fungsi, dan Contohnya dalam Pemrograman*. Lawencon. Tersedia pada: https://www.lawencon.com/rekursi-adalah/ (Diakses 14 April 2026).

GeeksforGeeks. 2025. *What is Tail Recursion*. Tersedia pada: https://www.geeksforgeeks.org/dsa/*tail-recursion*/ (Diakses 14 April 2026).

GeeksforGeeks. 2026. *What is the Call Stack in JavaScript*. Tersedia pada: https://www.geeksforgeeks.org/javascript/what-is-the-call-stack-in-javascript/ (Diakses 14 April 2026).

Program Studi Sarjana Informatika Telkom University. 2025. *Memahami Array: Menyimpan Banyak Data*. Tersedia pada: https://bif.telkomuniversity.ac.id/memahami-*array*-menyimpan-banyak-data/ (Diakses 14 April 2026).

Zulman, M. R. 2026. *Modul 3: Function & Array (Function, Arrow Function, Callback, Array & Method Array)*. Modul Praktikum Pemrograman JavaScript.